

```
import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.preprocessing import StandardScaler
```

```
np.random.seed(23) # Normally, random functions generate different numbers each time the program runs.
                   #By setting a seed value, the random sequence becomes reproducible.
```

```
mu_vec1 = np.array([0,0,0])
cov_mat1 = np.array([[1,0,0],[0,1,0],[0,0,1]])
class1_sample = np.random.multivariate_normal(mu_vec1, cov_mat1, 20)
```

```
mu_vec1 = np.array([0,0,0])
mu_vec1
```

```
array([0, 0, 0])
```

```
cov_mat1 = np.array([[1,0,0],[0,1,0],[0,0,1]])
cov_mat1
```

```
array([[1, 0, 0],
       [0, 1, 0],
       [0, 0, 1]])
```

`np.random.multivariate_normal()` is a function in NumPy used to generate random samples from a multivariate normal (Gaussian) distribution.

A multivariate normal distribution means a normal distribution with multiple variables that may be correlated with each other.

mean A list or array representing the mean values of each variable.

cov The covariance matrix, which describes the variance of each variable and the relationship between variables.

size The number of random samples to generate.

Uni-variate → one variable (example: only height)

Multi-variate → multiple variables (example: height and weight)

Multivariate distribution also considers the relationship (covariance) between variables.

```
class1_sample = np.random.multivariate_normal(mu_vec1, cov_mat1, 20)
```

```
class1_sample
```

```
array([[ 0.25073655, -0.81361625,  0.70362363],
       [-0.26814214, -0.48255948,  1.24461048],
       [ 0.67686012,  3.18750269, -1.08056475],
       [ 0.0102293 ,  0.43782968,  1.32778762],
       [-0.25114503,  1.59311063,  0.17081771],
       [-0.7092539 , -0.13302459, -0.01735694],
       [-0.10109291, -0.56403963, -0.1790356 ],
       [ 1.01105883,  0.92099584,  1.93308983],
       [-0.79536322, -1.0115352 ,  2.15077995],
```

```
[ 0.42514008,  0.44115151, -0.81743933],  
[ 0.43789248,  0.09972328,  0.06540607],  
[ 1.22443067, -0.76959905,  0.19211968],  
[-1.72325306,  0.4612592 , -1.08536678],  
[ 1.82337823, -1.33286349,  1.63739122],  
[-0.82293868, -1.59810911,  0.22651174],  
[ 0.23389799, -0.94722245, -1.26157621],  
[-2.38986578, -0.33327399,  0.34351676],  
[-2.02786082,  0.13141631,  1.60323411],  
[-1.76431364,  0.56650393,  0.54878807],  
[-0.61513498,  0.32354564, -1.10319346]])
```

```
type(class1_sample)
```

```
numpy.ndarray
```

```
class1_sample.ndim
```

```
2
```

```
class1_sample.shape
```

```
(20, 3)
```

```
df = pd.DataFrame(class1_sample, columns= ['feature1', 'feature2', 'feature3'])
```

```
df.head()
```

	feature1	feature2	feature3	
0	0.250737	-0.813616	0.703624	
1	-0.268142	-0.482559	1.244610	
2	0.676860	3.187503	-1.080565	
3	0.010229	0.437830	1.327788	
4	-0.251145	1.593111	0.170818	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df['target'] = 1
```

```
df.head()
```

	feature1	feature2	feature3	target	
0	0.250737	-0.813616	0.703624	1	
1	-0.268142	-0.482559	1.244610	1	
2	0.676860	3.187503	-1.080565	1	
3	0.010229	0.437830	1.327788	1	
4	-0.251145	1.593111	0.170818	1	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
mu_vec2 = np.array([1,1,1])
mu_vec2
```

```
array([1, 1, 1])
```

```
cov_mat2 = np.array([[1,0,0],[0,1,0],[0,0,1]])
cov_mat2
```

```
array([[1, 0, 0],
       [0, 1, 0],
       [0, 0, 1]])
```

```
class2_sample = np.random.multivariate_normal(mu_vec2, cov_mat2, 20)
class2_sample
```

```
array([[ 1.45059596,  2.61745203,  0.65763506],
       [ 0.37651362, -1.01480815,  2.87089397],
       [ 2.8731981 ,  0.49854403,  1.09009394],
       [ 0.90605542,  2.36048482, -0.56984254],
       [ 1.67640316, -1.25588661,  1.65361599],
       [ 0.2541872 ,  2.30678324,  1.54931008],
       [ 0.38781938,  0.26315789, -0.19535974],
       [ 2.68204548,  2.0654228 ,  0.3944364 ],
       [-0.33355413,  1.49605774,  1.01319381],
       [ 0.51478995,  1.11583088,  1.66509522],
       [ 0.9734437 ,  1.62321928,  0.99625858],
       [ 1.27196926,  0.45836122,  1.79379376],
       [ 1.15601287,  0.69540242,  1.18985569],
       [ 0.76793423,  0.80964176,  1.52538404],
       [ 1.87305874, -0.63096187,  0.80994688],
       [ 1.08129034, -0.11984219,  1.43735754],
       [ 0.76107671,  0.46260053, -0.76885535],
       [-0.44383133,  1.09345025,  2.79300305],
       [ 0.5829538 , -1.09724637,  2.23891913],
       [ 1.17291706,  2.1717961 ,  0.37141921]])
```

```
type(class1_sample)
```

```
numpy.ndarray
```

```
class1_sample.ndim
```

```
2
```

```
class1_sample.shape
```

```
(20, 3)
```

```
df1 = pd.DataFrame(class2_sample,columns=['feature1','feature2','feature3'])
```

```
df1.head()
```

	feature1	feature2	feature3	
0	1.450596	2.617452	0.657635	
1	0.376514	-1.014808	2.870894	
2	2.873198	0.498544	1.090094	
3	0.906055	2.360485	-0.569843	
4	1.676403	-1.255887	1.653616	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
df1['target'] = 0
df1.head()
```

	feature1	feature2	feature3	target	
0	1.450596	2.617452	0.657635	0	
1	0.376514	-1.014808	2.870894	0	
2	2.873198	0.498544	1.090094	0	
3	0.906055	2.360485	-0.569843	0	
4	1.676403	-1.255887	1.653616	0	

Next steps: [Generate code with df1](#) [New interactive sheet](#)

```
df2 = pd.concat([df, df1], ignore_index=True)
df2.head()
```

	feature1	feature2	feature3	target	
0	0.250737	-0.813616	0.703624	1	
1	-0.268142	-0.482559	1.244610	1	
2	0.676860	3.187503	-1.080565	1	
3	0.010229	0.437830	1.327788	1	
4	-0.251145	1.593111	0.170818	1	

Next steps: [Generate code with df2](#) [New interactive sheet](#)

```
df2.shape
```

```
(40, 4)
```

```
df2.ndim
```

```
2
```

```
df2.sample(10)
```

	feature1	feature2	feature3	target
7	1.011059	0.920996	1.933090	1
27	2.682045	2.065423	0.394436	0
26	0.387819	0.263158	-0.195360	0
9	0.425140	0.441152	-0.817439	1
35	1.081290	-0.119842	1.437358	0
14	-0.822939	-1.598109	0.226512	1
15	0.233898	-0.947222	-1.261576	1
21	0.376514	-1.014808	2.870894	0
6	-0.101093	-0.564040	-0.179036	1
22	2.873198	0.498544	1.090094	0

df2.info()

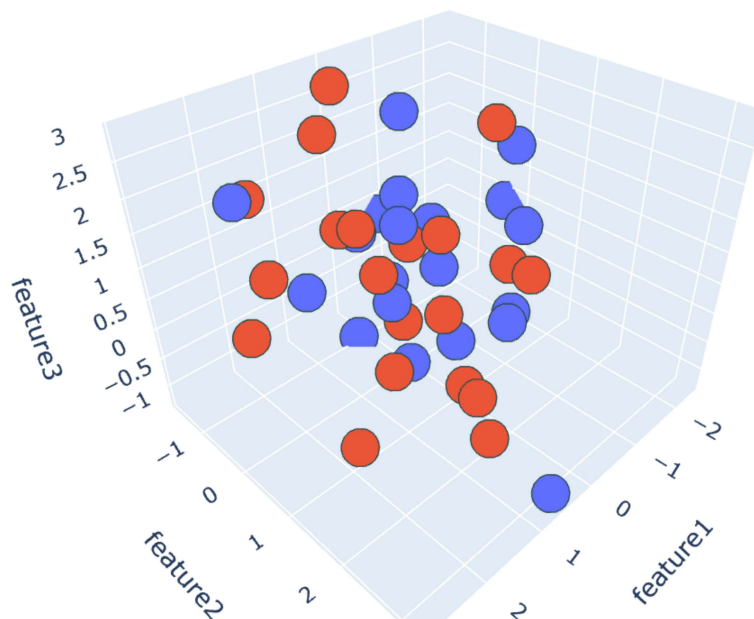
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   feature1    40 non-null     float64
1   feature2    40 non-null     float64
2   feature3    40 non-null     float64
3   target      40 non-null     int64
dtypes: float64(3), int64(1)
memory usage: 1.4 KB
```

df2.describe()

	feature1	feature2	feature3	target
count	40.000000	40.000000	40.000000	40.00000
mean	0.365253	0.402416	0.727982	0.50000
std	1.168812	1.203103	1.087826	0.50637
min	-2.389866	-1.598109	-1.261576	0.00000
25%	-0.284495	-0.580770	0.044715	0.00000
50%	0.431516	0.439491	0.756785	0.50000
75%	1.099971	1.099045	1.562791	1.00000
max	2.873198	3.187503	2.870894	1.00000

```
fig = px.scatter_3d(df2, x=df2['feature1'], y=df2['feature2'], z=df2['feature3'],
                    color=df2['target'].astype('str'))
fig.update_traces(marker=dict(size=12,
                              line=dict(width=2,
                                          color='DarkSlateGrey')),
                  selector=dict(mode='markers'))

fig.show()
```



Step 1 - Apply standard scaling

```
scaler = StandardScaler()
```

```
df2.iloc[:,0:3] = scaler.fit_transform(df2.iloc[:,0:3])
```

Step 2 - Find Covariance Matrix

```
covariance_matrix = np.cov([df2.iloc[:,0],df2.iloc[:,1],df2.iloc[:,2]])
print('Covariance Matrix:\n', covariance_matrix)
```

Covariance Matrix:

```
[[ 1.02564103  0.17616201  0.13967602]
 [ 0.17616201  1.02564103 -0.19226931]
 [ 0.13967602 -0.19226931  1.02564103]]
```

Step 3 - Finding eigen_values and eigen_vectors

```
eigen_values, eigen_vectors = np.linalg.eig(covariance_matrix)
```

eigen_values

```
array([0.68598402, 1.16408842, 1.22685063])
```

eigen_vectors

```
array([[ -0.55061735,  0.77018875, -0.32191587],
       [ 0.60884476,  0.10671298, -0.78607913],
       [ 0.5710767 ,  0.62882559,  0.5276834 ]])
```

```

%pylab inline

from matplotlib import pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d import proj3d
from matplotlib.patches import FancyArrowPatch

class Arrow3D(FancyArrowPatch):

    def __init__(self, xs, ys, zs, *args, **kwargs):
        super().__init__((0,0),(0,0), *args, **kwargs)
        self._verts3d = xs, ys, zs

    def do_3d_projection(self, renderer=None):
        xs3d, ys3d, zs3d = self._verts3d
        return min(zs3d)

    def draw(self, renderer):
        xs3d, ys3d, zs3d = self._verts3d
        xs, ys, zs = proj3d.proj_transform(xs3d, ys3d, zs3d, self.axes.get_proj())
        self.set_positions((xs[0], ys[0]), (xs[1], ys[1]))
        super().draw(renderer)

fig = plt.figure(figsize=(7,7))
ax = fig.add_subplot(111, projection='3d')

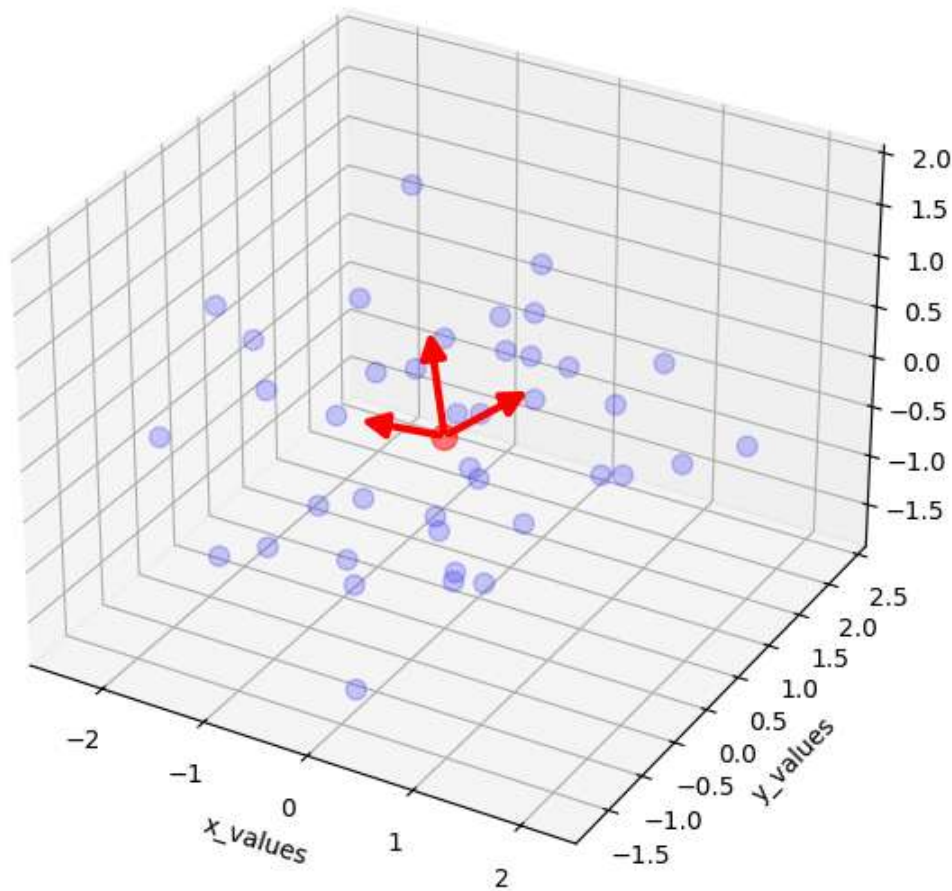
ax.plot(df2['feature1'], df2['feature2'], df2['feature3'], 'o', markersize=8, color='blue', alpha=0.2)
ax.plot([df2['feature1'].mean()], [df2['feature2'].mean()], [df2['feature3'].mean()], 'o', markersize=8)
for v in eigen_vectors.T:
    a = Arrow3D([df2['feature1'].mean(), v[0]], [df2['feature2'].mean(), v[1]], [df2['feature3'].mean(), v[2]])
    ax.add_artist(a)
ax.set_xlabel('x_values')
ax.set_ylabel('y_values')
ax.set_zlabel('z_values')

plt.title('Eigenvectors')

plt.show()

```

Eigenvectors



```
pc = eigen_vectors[0:2]
pc
```

```
array([[ -0.55061735,  0.77018875, -0.32191587],
       [ 0.60884476,  0.10671298, -0.78607913]])
```

```
transformed_df = np.dot(df2.iloc[:,0:3],pc.T)
```

```
transformed_df
```

```
array([[ -0.72644776, -0.15182048],
       [ -0.42639374, -0.79171931],
       [  2.19898693,  1.73809686],
       [  0.01257982, -0.62305969],
       [  1.23301499,  0.18952411],
       [  0.38887562, -0.06949449],
       [ -0.13225673,  0.33094036],
       [ -0.33306709, -0.49464732],
       [ -0.78938304, -1.7805222 ],
       [  0.45969766,  1.16604316],
       [ -0.0323275 ,  0.49601675],
       [ -1.00915743,  0.74013005],
       [  1.57801388,  0.2305496 ],
       [ -2.09323061, -0.05217382],
       [ -0.57982212, -0.43954212],
       [ -0.21607244,  1.26546763],
       [  0.95270491, -1.23817589],
```



```
[ 0.70373296, -1.92734617],
[ 1.17608715, -0.97756519],
[ 0.96539812,  0.81580736],
[ 0.9393345 ,  0.82302131],
[-1.5664142 , -1.68959214],
[-1.24272406,  1.06668864],
[ 1.40040265,  1.4109615 ],
[-1.97806636, -0.13466874],
[ 1.04148748, -0.48859091],
[ 0.175671   ,  0.67511561],
[ 0.07280428,  1.61569294],
[ 0.9569536 , -0.47913629],
[ 0.11033237, -0.54282607],
[ 0.42091145,  0.23418038],
[-0.71573635, -0.29662319],
[-0.32573723,  0.10547157],
[-0.16707992, -0.33454195],
[-1.41389162,  0.64262581],
[-0.89280557, -0.18830665],
[ 0.29877017,  1.30963691],
[ 0.21514483, -1.87597819],
[-1.52895191, -1.12559897],
[ 0.86866133,  0.84595923]])
```

```
df2.shape
```

```
(40, 4)
```

```
transformed_df.shape
```

```
(40, 2)
```

```
# 40,3 - 3,2
```

```
new_df = pd.DataFrame(transformed_df,columns=['PC1','PC2'])
```

```
new_df.sample(5)
```

	PC1	PC2
19	0.965398	0.815807
26	0.175671	0.675116
11	-1.009157	0.740130
24	-1.978066	-0.134669
17	0.703733	-1.927346

```
new_df['target'] = df2['target'].values
new_df.sample(5)
```

PC1

PC2 target



```
new_df['target'] = new_df['target'].astype('str')
```

```
20 0.958934 -0.479130 0
```

```
fig = px.scatter(x=new_df['PC1'],
                 y=new_df['PC2'],
                 color=new_df['target'],
                 color_discrete_sequence=px.colors.qualitative.G10
                 )
```

```
fig.update_traces(marker=dict(size=12,
                               line=dict(width=2,
                                           color='DarkSlateGrey')),
                  selector=dict(mode='markers'))
```

```
fig.show()
```

